

Dynamic Implementation of Stack:

```
#include <stdio.h>

#include <stdlib.h>

struct Node

{

    int data;

    struct Node *next;

};

struct Node *top = NULL;

void push(int value)

{

    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if(newNode == NULL)

    {

        printf("Stack Overflow\n");

        return;

    }

    newNode->data = value;

    newNode->next = top;

    top = newNode;

    printf("%d inserted into stack.\n", value);

}

void pop()

{

    struct Node *temp;

    if(top == NULL)

    {
```

```

        printf("Stack Underflow\n");
        return;
    }
    temp = top;
    printf("%d deleted from stack.\n", top->data);
    top = top->next;
    free(temp);
}

void peek()
{
    if(top == NULL)
        printf("Stack is empty.\n");
    else
        printf("Top element = %d\n", top->data);
}

void display()
{
    struct Node *temp;
    if(top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements are:\n");
    temp = top;
    while(temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

```

```
}  
  
int main()  
{  
    int choice, value;  
    while(1)  
    {  
        printf("\n--- STACK MENU ---\n");  
        printf("1. Push\n");  
        printf("2. Pop\n");  
        printf("3. Peek\n");  
        printf("4. Display\n");  
        printf("5. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
        switch(choice)  
        {  
            case 1:  
                printf("Enter value: ");  
                scanf("%d", &value);  
                push(value);  
                break;  
            case 2:  
                pop();  
                break;  
            case 3:  
                peek();  
                break;  
            case 4:  
                display();  
                break;
```

```

        default:

            return 0;

        }

    }

}

```

Static Implementation of Stack

```

#include<stdio.h>

int stack[100];
int top=0;

void push(int value) {
    if (top >= 100) {
        printf("Stack overflow\n");
        return;
    }
    stack[top++] = value;
}

int pop() {
    if (top <= 0) {
        printf("Stack underflow\n");
        return -1;
    }
    return stack[--top];
}

void isEmpty() {
    if (top <= 0) {
        printf("Stack is empty\n");
    } else {
        printf("Stack is not empty\n");
    }
}

```

```

    }
}

int peek() {
    if (top <= 0) {
        printf("Stack is empty\n");
        return -1;
    }
    return stack[top-1];
}

int main() {
    int choice, value;
    while (1) {
        printf("1. Push\n2. Pop\n3. Peek\n4. Is Empty\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                value = pop();
                if (value != -1) printf("Popped value: %d\n", value);
                break;
            case 3:
                value = peek();
                if (value != -1) printf("Top value: %d\n", value);
                break;
            case 4:

```

```
        isEmpty();  
        break;  
    default:  
        return 0;  
    }  
}  
return 0;  
}
```